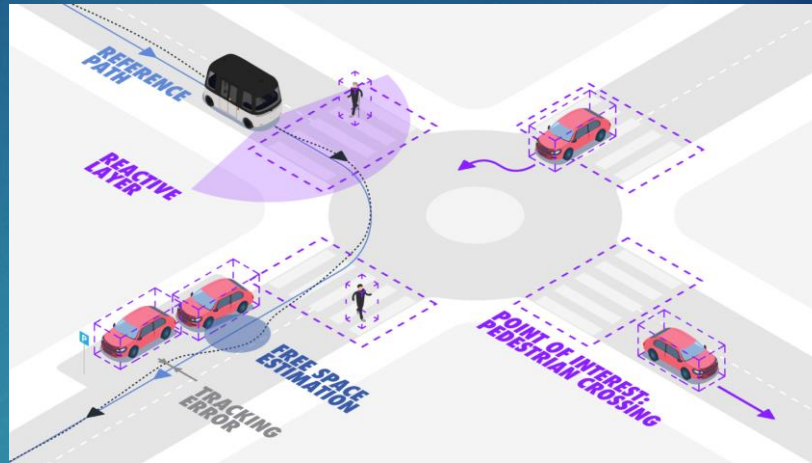


[<https://sensible4.fi>]



TIPE

Génération et suivi de
trajectoire par une voiture
autonome dans un milieu
urbain

LÉRONDEL—TOUZÉ ADAM – CANDIDAT N°23764

THÈME : LA VILLE

Table des matières

2

Adam LÉRONDEL--TOUZÉ

- ▶ Contexte et problématique
- ▶ Mes objectifs
- ▶ 1. Méthode des tentacules
- ▶ 2. Implémentation en Python
- ▶ 3. Critiques, tests et ouvertures
- ▶ Conclusion

Contexte et problématique

3

Adam LÉRONDEL--TOUZÉ



[entreprises.gouv.fr]

Mes Objectifs

4

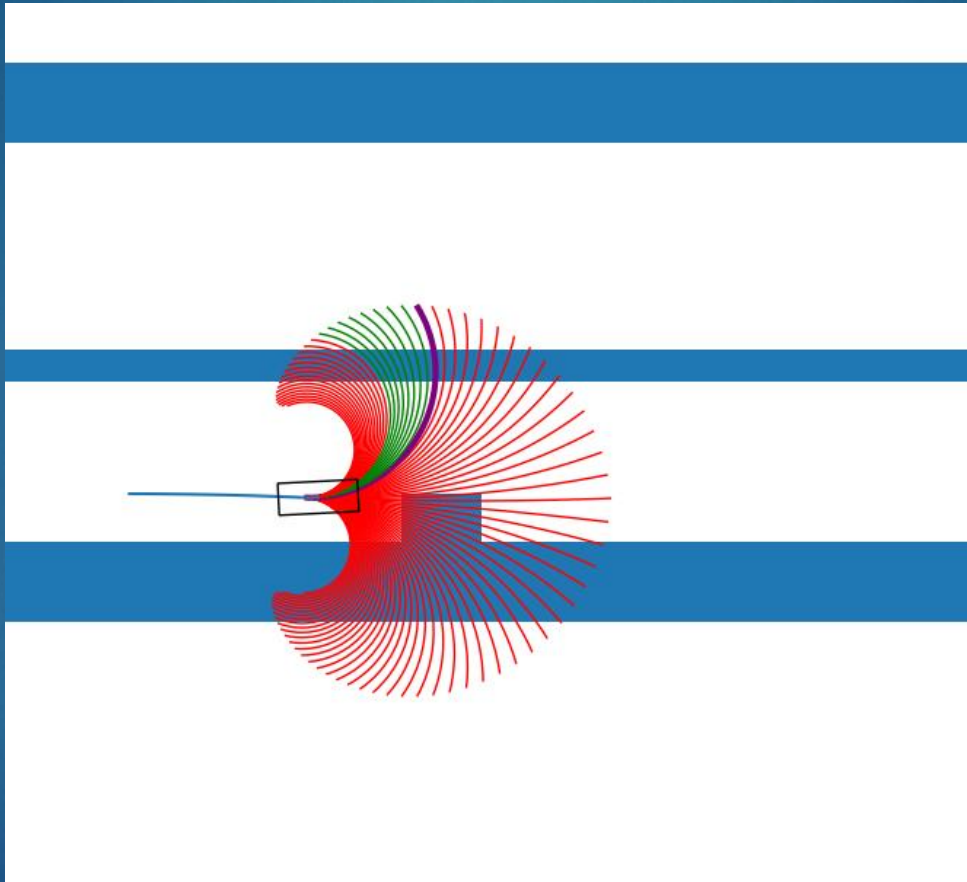
Adam LÉRONDEL-TOUZÉ

- ▶ Algorithme → Python
- ▶ Affiner un modèle
- ▶ Clothoïde
- ▶ Expérimenter, discuter

1. Méthode des tentacules

5

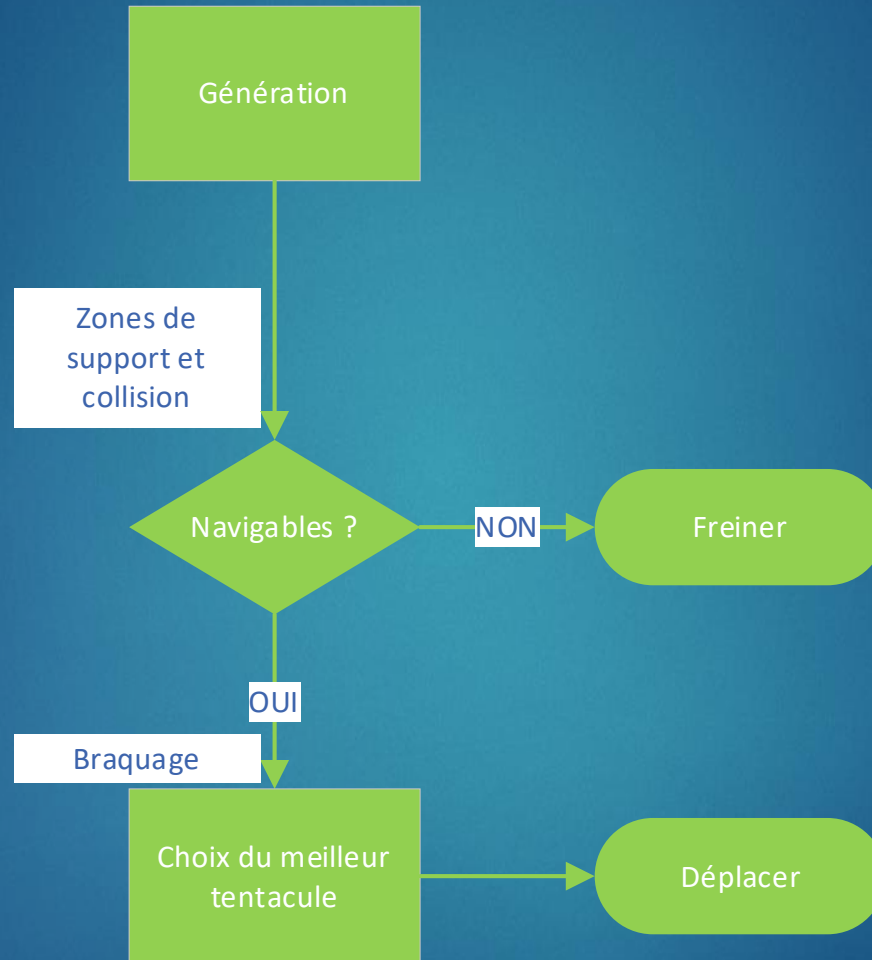
Adam LÉRONDEL--TOUZÉ



Algorithme

6

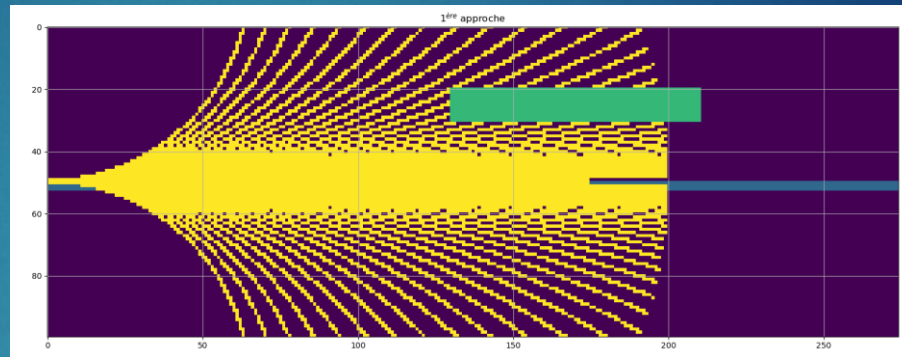
1. Méthode des tentacules



2. Implémentation en Python

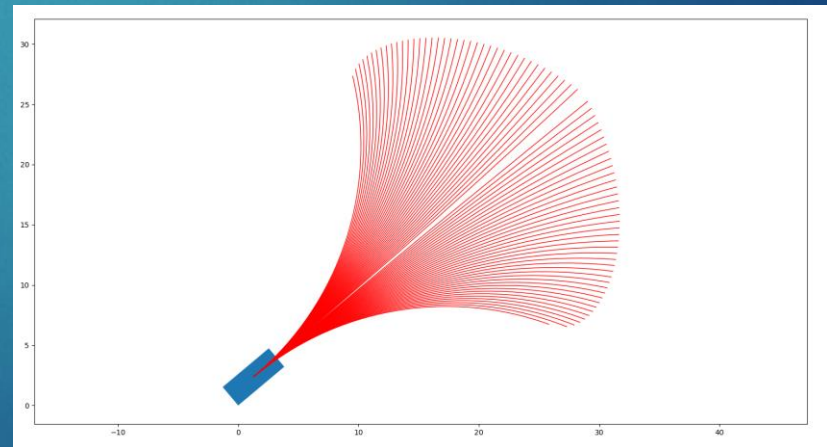
7

1^{ère} approche



Tentacules, pixels manuels

Choix de « boîtes noires » Matplotlib



Tentacules, objets Matplotlib

Structure adoptée

Fonctionnement

- ▶ Boucle pour l'algorithme
- ▶ Boucle d'affichage
- ▶ Boucle de « contrôle »

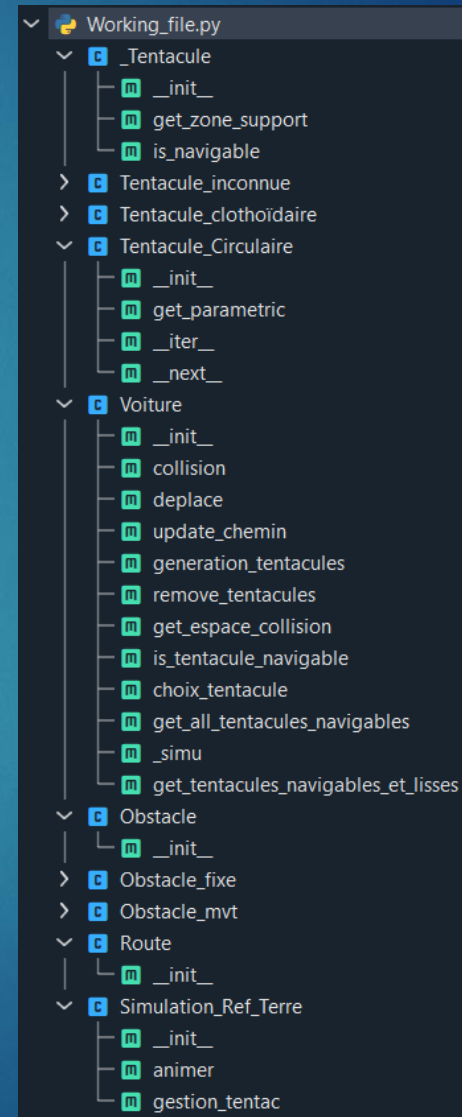
Structure adoptée

9

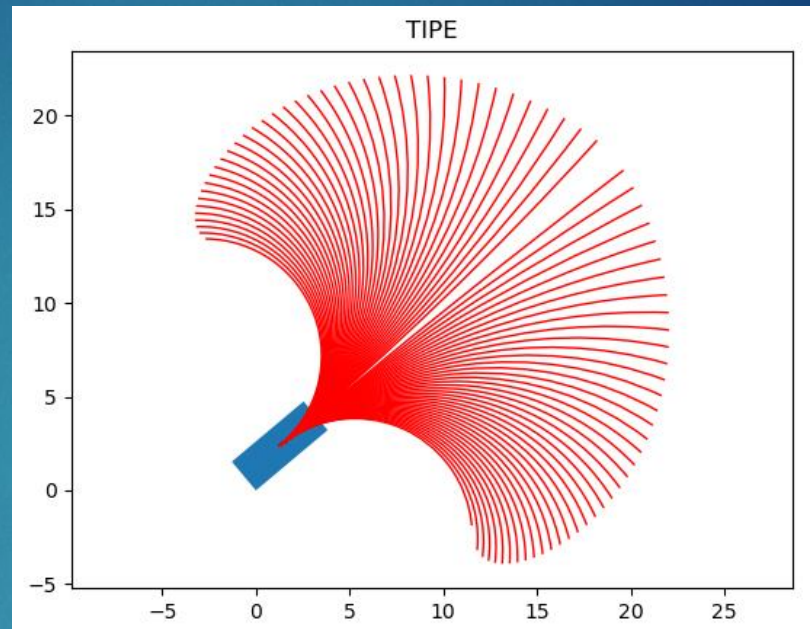
Organisation

- ▶ POO
- ▶ Génération
- ▶ Déplacement
- ▶ Collision
- ▶ Navigation
- ▶ Optimisation
- ▶ Clothoïdes

2. Implémentation en Python



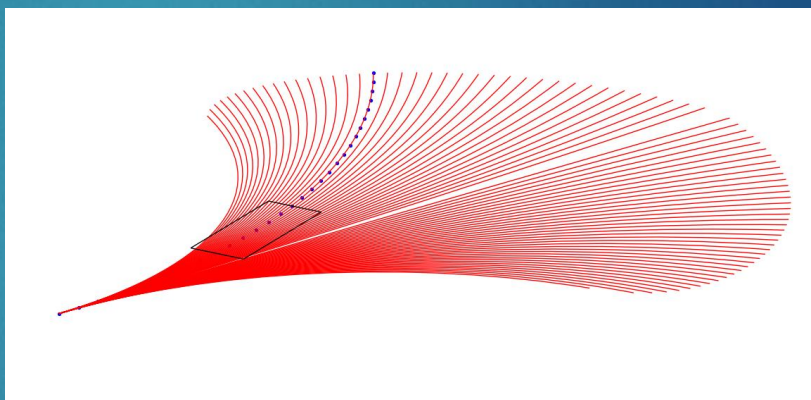
Génération



```
# On créé le dessin de la tentacule comme si la voiture était orientée vers le Nord
self.patch = matplotlib.patches.Arc((centre_x, self.voiture.y), self.rayon, self.rayon,
                                     theta1=self.angle_debut, theta2=self.angle_fin, color='pink')
# On applique la rotation de la voiture au dessin de la tentacule
self.patch.set_transform(matplotlib.transforms.Affine2D().rotate_deg_around(self.voiture.x,
                                                                            self.voiture.y,
                                                                            self.voiture.angle - 90)
                        + self.axes.transData)
```

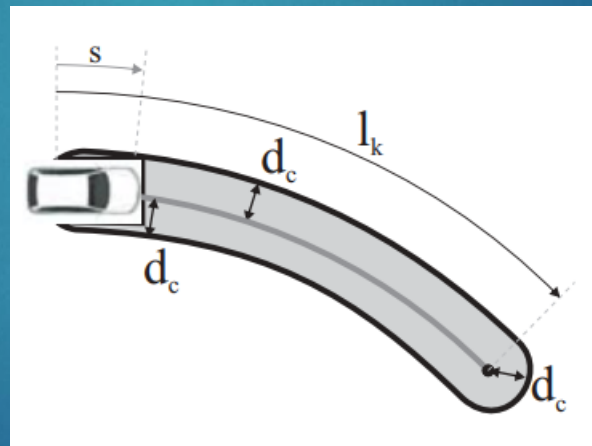
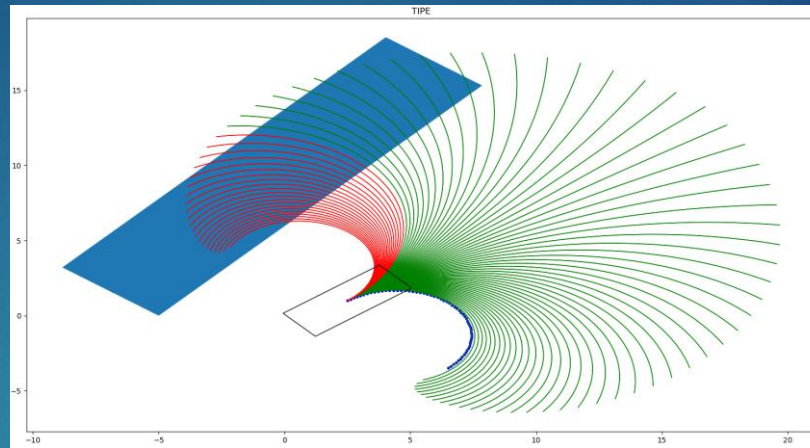
Déplacement

- `get_parametric()`
- Dérivée discrète



Collision

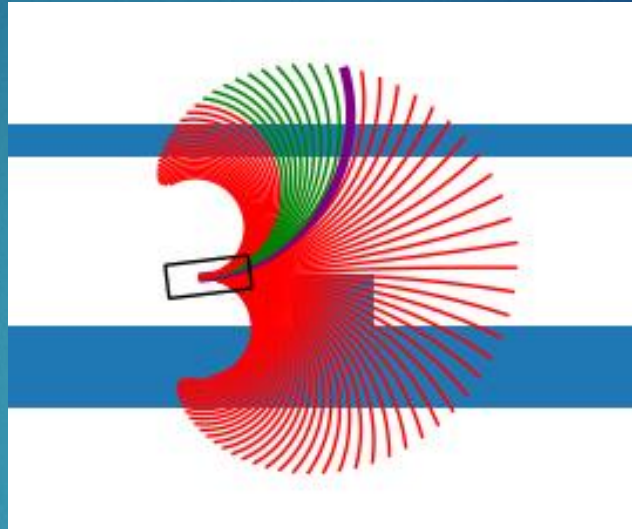
- Distance collision



[von Hundelshausen et al]

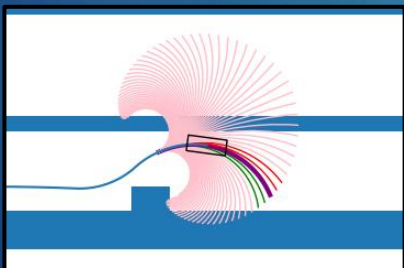
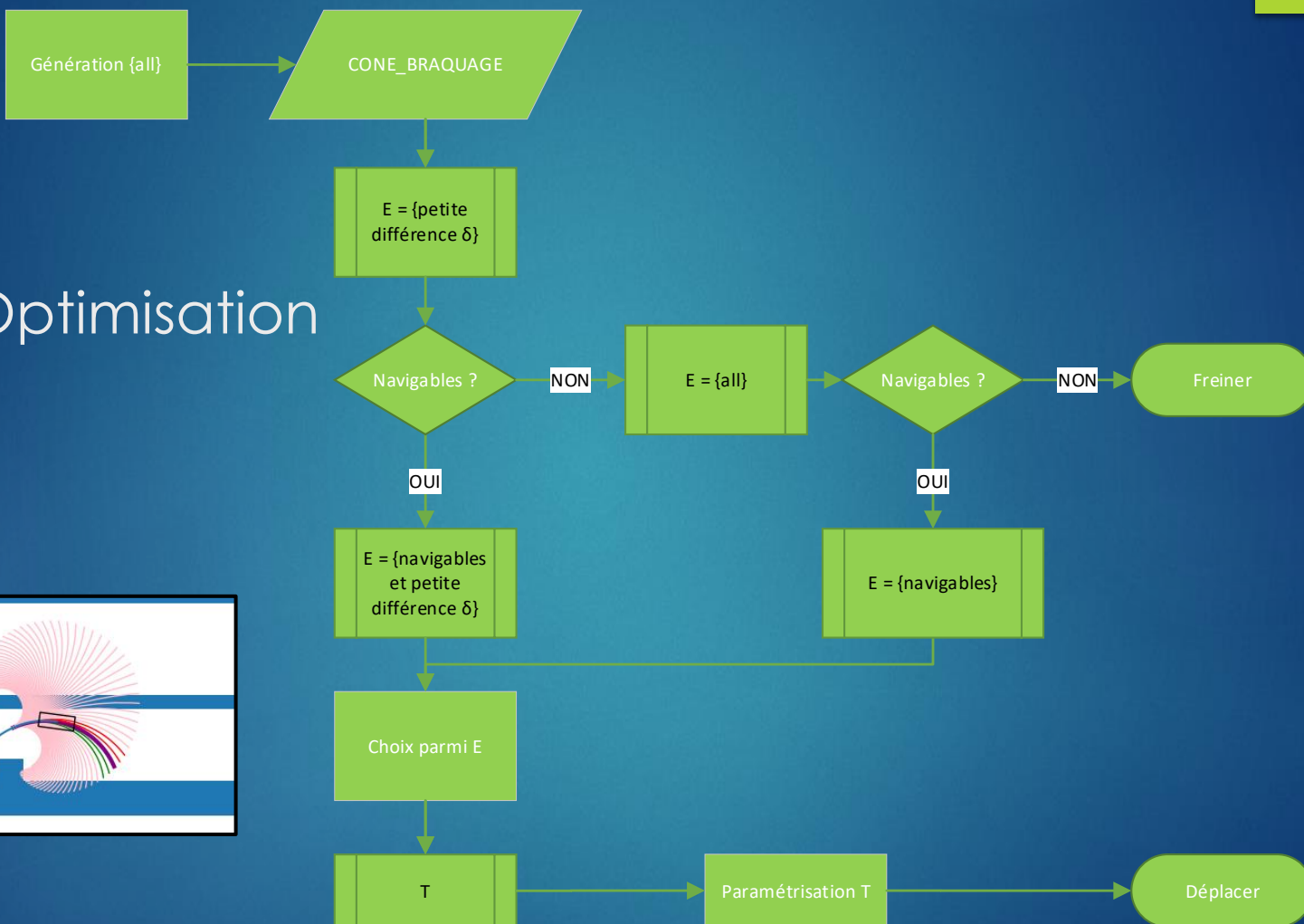
Navigation

- Fonction de tri



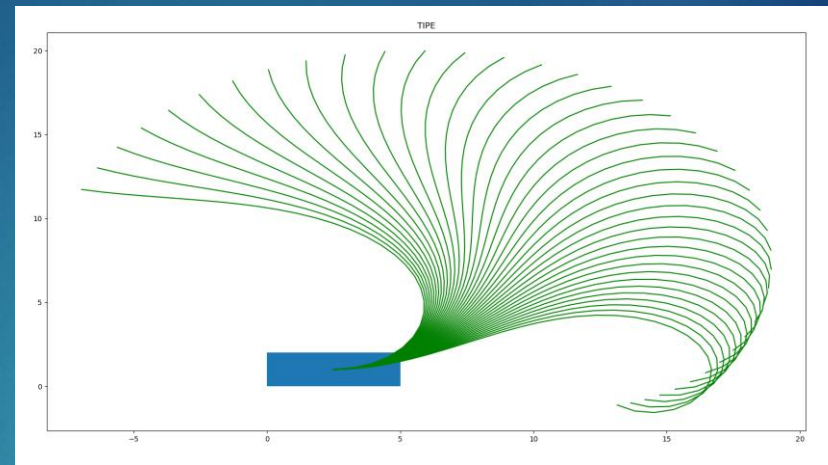
```
return min(tentacules_navigables, key=lambda x: x.V_lisse**4 + x.braquage**2)
```

Optimisation



Clothoïdes

- Intégrales de Fresnel :
$$\begin{cases} x(t) = \int_0^t [\cos(as^2 + bs)] ds \\ y(t) = \int_0^t [\sin(as^2 + bs)] ds \end{cases}$$
- Objet Tentacule



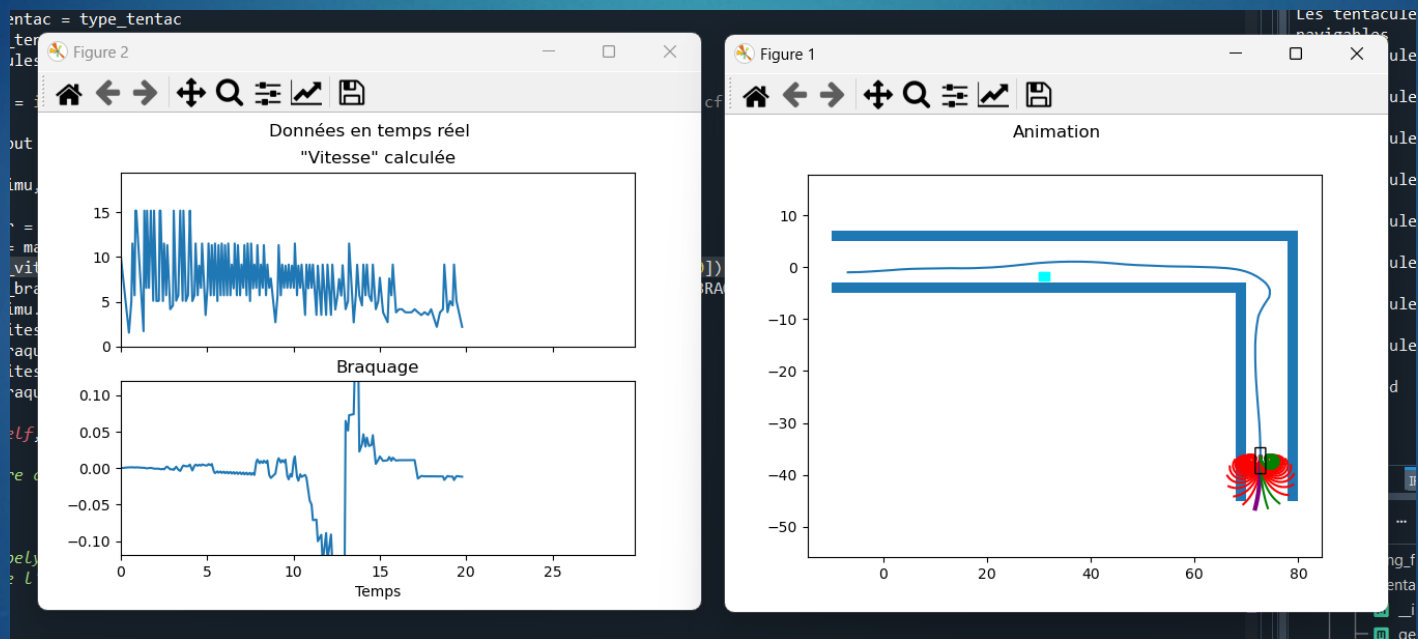
```
▼ c _Tentacule
  │
  │ m _init_
  │ m get_zone_support
  │ m is_navigable
  │
  ▼ c Tentacule_clothoïdaire
    │
    │ m _init_
    │ m x
    │ m y
    │ m get_parametric
    │ m _iter_
    │ m _next_
    │
    > c Tentacule_Circulaire
```

Tests et critiques

16

Adam LÉRONDEL--TOUZÉ

- Programme satisfaisant et résultats interprétables



Tests et critiques

17

Tests et critiques

- ▶ Résultats confirmés :
 - ▶ Trajectoire plus naturelle si clothoïdes
 - ▶ Choix de la POO judicieux : temps de calcul raisonnable

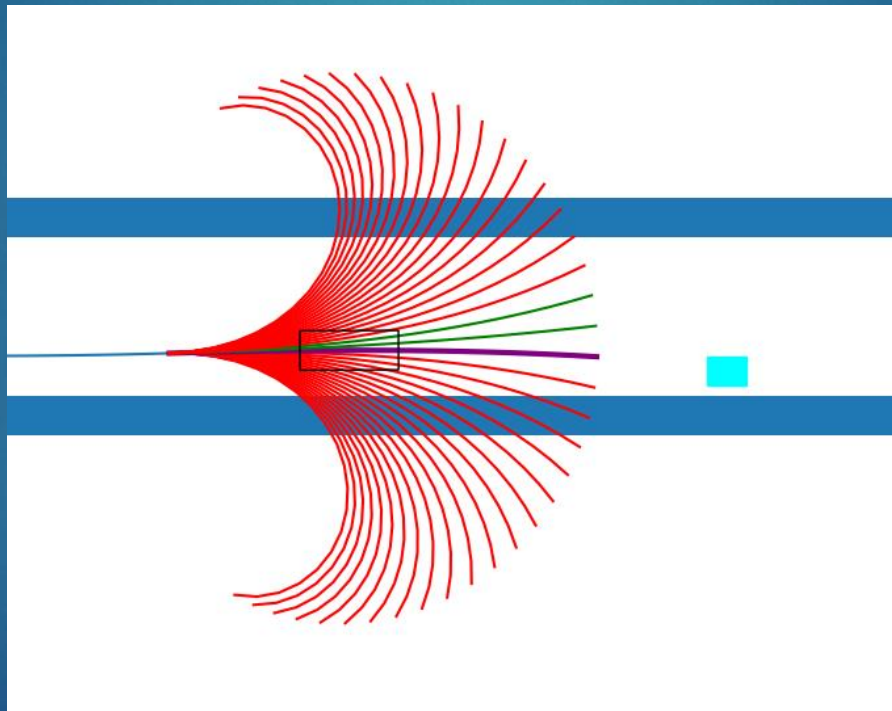


Tests et critiques

18

Tests et critiques

- Délai entre les générations crucial



Tests et critiques

19

Tests et critiques

- ▶ Variation des paramètres

Ouvertures

20

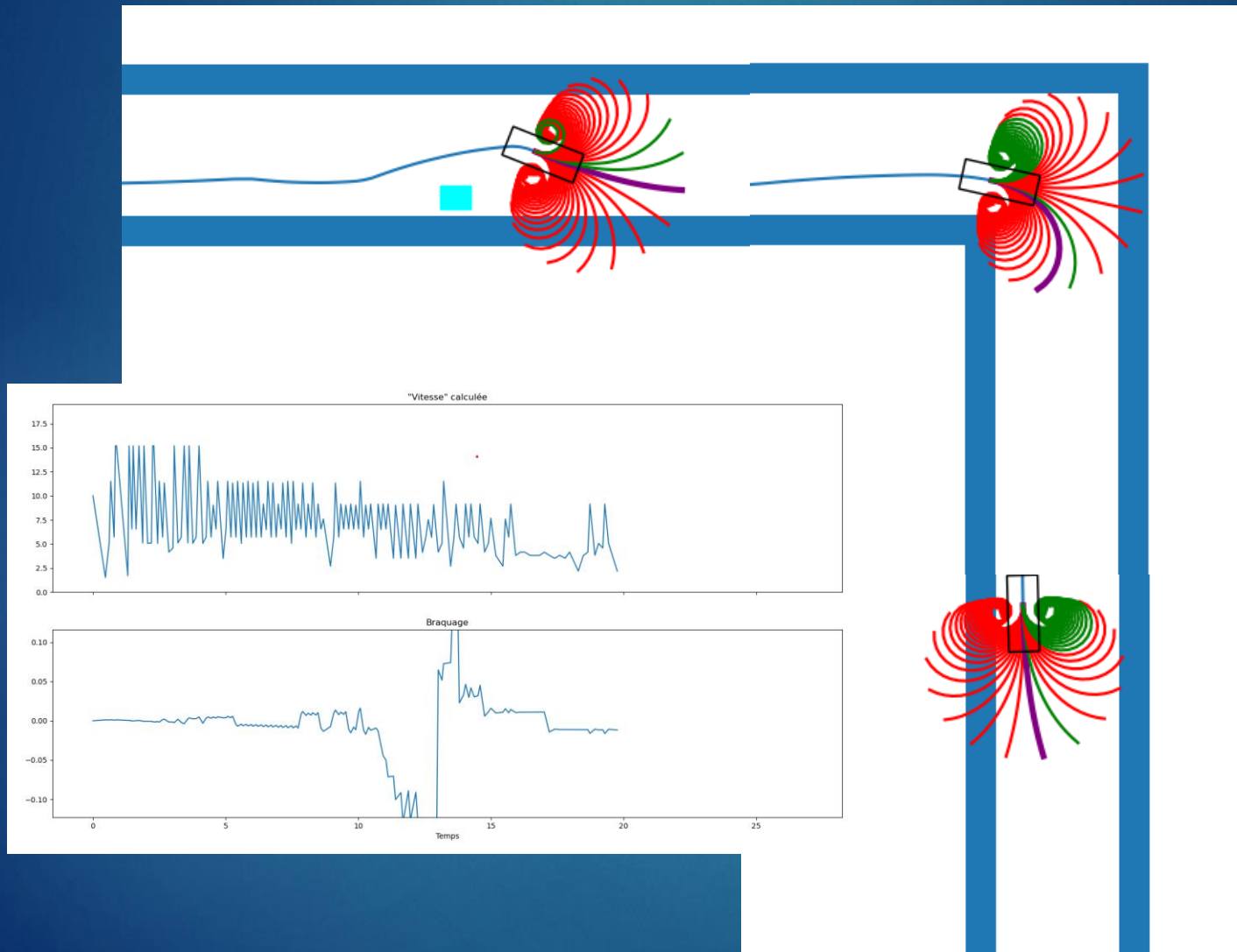
Adam LÉRONDEL--TOUZÉ

- ▶ Plusieurs voitures
- ▶ Obstacles en mouvement

Conclusion

21

Adam LÉRONDEL--TOUZÉ



```

1  # -*- coding: utf-8 -*-
2  """
3  Created on Mon Dec 29 19:50:23 2022
4
5  @author: adaml
6  """
7
8
9  import numpy as np
10 import matplotlib.pyplot as plt
11 import matplotlib.animation
12 import matplotlib.patches
13 import matplotlib.transforms
14 import matplotlib.lines
15 import scipy.interpolate
16 import shapely
17 import shapely.geometry
18 import time
19 from scipy.integrate import quad as integrale
20
21
22 VITESSE_INIT_VOIT = 10 # vitesse initiale du véhicule
23 VITESSE_MAX_VOIT = 15 # vitesse maximale (pour rester dans les zones linéaires des modèles)
24 ACCEL_LAT_MAX = 1.5 # accélération latérale maximale (pour rester dans les zones linéaires des modèles)
25 LONGUEUR_VEHIC = 5 # longueur du véhicule
26 LARGEUR_VEHIC = 2 # largeur du véhicule
27 POSITION_VOITURE_INIT = (-10, -2) # position (x, y) initiale de la voiture
28 INCLINAISON_VOITURE_INIT = 0 # inclinaison initiale de la voiture (en degrés)
29 BRAQUAGE_VOLANT_INIT = 0 # angle volant/voiture en degrés
30 PRECISION = 20 # nombre de points définissant l'arc de cercle. ie nombre de positions où la voiture ira
31 TEMPS_REAC = 1200 # temps de génération de tentacules
32 CONE_BRAQUAGE = 0.02 # valeur de braquage définissant des trajectoires "lisses". cf get_tentacules_navigables_et_lisses
33 COMMENTAIRES = True # si des commentaires sur l'évolution de l'algo doivent être affichés dans la console
34 PETIT_POUCE = True # si on veut récupérer la trajectoire de la voiture
35 SECURITE = True # vérifie les collisions. False si aucune sécu, None si check sans arrêt True si arrêt lorsque collision
36
37 # courbure = r0
38 # courbure = 1/rayon dans le cas d'un cercle
39 # braquage = delta = arctan(L/rayon) quasi logique (schéma + perpendiculaire)
40 # longueur = angle*rayon
41 # empattement += longueur voiture
42
43
44 class _Tentacule():
45     """
46     Classe dont héritent tout type de tentacule. Elle rassemble des méthodes communes universelles. N'est pas censée être utilisée directement ; elle est appelée automatiquement par d'autres instances.
47     """
48
49     def __init__(self, axes, voiture):
50         """
51         Parameters
52         -----
53         axes : matplotlib.axes._axes.Axes
54             Axes dans lesquels la tentacule sera affichée.
55         voiture : __main__.Voiture

```

```

56         Voiture à laquelle appartient la tentacule.
57
58         Returns
59         -----
60         None.
61
62         """
63         self.axes = axes
64         self.voiture = voiture
65         self.precision = PRECISION
66
67     def get_zone_support(self):
68         """
69         Génère et stocke une zone de support pour la tentacule (distance de sécurité autour de la tentacule pour éviter quelque collision). cf von Hundelshausen et al 2008 p.647
70
71         Raises
72         -----
73         ValueError
74             Si la vitesse du véhicule est trop grande (55km/h).
75
76         Returns
77         -----
78         shapely.geometry.polygon.Polygon
79             La zone de support.
80
81         """
82         if self.voiture.vitesse < 3:
83             self.dist_zone_support = 1.4 + 0.2*self.voiture.vitesse/3 # formule empirique cf p.123 eq(9.6) thèse
84         elif 3 <= self.voiture.vitesse <= 15:
85             self.dist_zone_support = 1.6 + 0.6 * (self.voiture.vitesse-3) / 15 # idem
86         else:
87             raise ValueError("La vitesse est trop élevée !")
88         self.zone_support = shapely.geometry.LineString(self.get_parametric()).buffer(self.dist_zone_support)
89         return self.zone_support
90
91     def is_navigable(self, objets_alentours):
92         """
93         Navigabilité de la tentacule.
94
95         Parameters
96         -----
97         objets_alentours : __main__.Voiture ou __main__.Obstacle
98             Objet ayant un attribut 'shapely_poly'.
99
100
101         Returns
102         -----
103         bool
104             True si la tentacule est navigable, False sinon.
105
106         """
107         self.zone_to_be_clean = self.get_zone_support().intersection(self.voiture.get_espace_collision())
108         for objet in objets_alentours:
109             if objet == self.voiture: # Pas grave si la tentacule cogne sa propre voiture pass
110                 pass
111             elif self.zone_to_be_clean.intersection(objet.shapely_poly).is_empty is False:
112                 self.patch.set_color("red")
113                 return False

```

```

114         return True
115
116
117 class Tentacule_inconnue(_Tentacule):
118     """
119     Tentacule space inconnue au bataillon. Fonctionnalités pas nécessaire pour notre étude
    ↪ donc classe aux fonctionnalités pas totalement développées.
120     """
121
122     def __init__(self, axes, voiture, patch, braquages):
123         """
124         Parameters
125         -----
126         axes : matplotlib.axes._axes.Axes
127             Axes dans lesquels la tentacule sera affichée.
128         voiture : __main__.Voiture
129             Voiture à laquelle appartient la tentacule.
130         patch : matplotlib.patches.Patch
131             Forme de la tentacule (étant donné que sa caractérisation n'est pas explicite)
132         braquages : list ou itérable
133             Itérable contenant les braquages successifs que prendra la voiture.
134
135         Returns
136         -----
137         None.
138
139         """
140         super().__init__(axes, voiture)
141         self.patch = patch
142         self.braquages = braquages
143
144     def get_parametric(self):
145         """
146         Permet d'obtenir la forme paramétrée de la tentacule. Utilise une interpolation.
147
148         Returns
149         -----
150         chemin : numpy.ndarray
151             Abscisses et ordonnées de la tentacule.
152             Ex: numpy.array([[ x1,   y1],
153                             [ x2,   y2],
154                             [ x3,   y3]])
155
156         """
157         self.chemin = (self.patch.get_transform() -
    ↪ axes_simu.transData).transform_path(self.patch.get_path()).cleaned().vertices
158         # Interpolation du chemin
159         self.chemin_interpol = scipy.interpolate.interpdd(self.chemin[:, 0], self.chemin[:, 1])
    ↪ 1])
160         lch = len(self.chemin[:, 0])
161         # Interpolation de x(t) pour faire une génération d'abscisses supplémentaires
162         points = scipy.interpolate.interpdd(range(0, (lch)*self.precision, self.precision),
    ↪ self.chemin[:, 0]) # va jusque (lch-1)*precision inclus
163         # Génération des abscisses supplémentaires avec le points(range) et des ordonnées
    ↪ interpolées correspondantes avec le chemin_interpol(x)
164         chemin = np.array([x, self.chemin_interpol(x)] for x in points(range(0,
    ↪ (lch-1)*self.precision, lch-1))))
165         return chemin
166
167     def __iter__(self):
168         """
169         Implémente l'itérabilité.

```

```

170
171         self.chemin = self.get_parametric()
172         return self
173
174     def __next__(self):
175         """
176         Passe à l'élément suivant (la tentacule étant un itérable) : met à jour la forme
    ↪ paramétrée, et retourne les nouveaux états de la voiture.
177
178         Raises
179         -----
180         StopIteration
181             Si la tentacule est parcourue en entier.
182
183         Returns
184         -----
185         new_x : float
186             Nouvelle abscisse.
187         new_y : float
188             Nouvelle ordonnée.
189         dz : float
190             Déplacement effectué en x.
191         dy : float
192             Déplacement effectué en y.
193         new_angle : numpy.float64
194             Nouvel angle de la voiture.
195         braquage : numpy.float64 ou celui de la liste de braquage passée en paramètre lors
    ↪ de la définition de la tentacule.
196             Nouveau braquage de la voiture.
197
198         """
199         try:
200             new_x = self.chemin[1][0]
201             new_y = self.chemin[1][1]
202             dx = new_x - self.chemin[0][0]
203             dy = new_y - self.chemin[0][1]
204             new_angle = np.degrees(np.arctan2(dy, dx))
205             braquage = next(self.braquages)
206             self.chemin = np.delete(self.chemin, 0, 0)
207             return new_x, new_y, dx, dy, new_angle, braquage
208         except IndexError:
209             raise StopIteration("Fin de la tentacule")
210
211
212 class Tentacule_clothoïdaire(_Tentacule):
213     """
214     Tentacule en forme de clothoïde, dépendante d'une voiture et caractérisée par une
    ↪ courbure initiale et d'une variation de courbure.
215
216     """
217
218     def __init__(self, voiture, courbure_init, var_courbure, axes):
219         """
220         Parameters
221         -----
222         voiture : __main__.Voiture
223             Voiture à laquelle appartient la tentacule.
224         courbure_init : float
225             Courbure initiale de la tentacule.
226         var_courbure : float
227             Variation de la courbure. # TODO: clarifier avec Adrien
228         axes : matplotlib.axes._axes.Axes
229             Axes dans lesquels la tentacule sera affichée.

```

```

229 Returns
230 -----
231 None.
232
233
234
235 """
236 super().__init__(axes, voiture)
237
238 self.longueur = 7*self.voiture.vitesse - 5 # formule empirique, équation (9.15)
239 ↳ p.128
240 self.courbure_init = courbure_init
241 self.var_courbure = var_courbure
242 self.courb_finale = self.courbure_init + self.var_courbure*self.longueur
243
244 self.xdata = [self.x(t, self.var_courbure, self.courb_finale) for t in
245 ↳ np.linspace(0, int(self.longueur), self.precision)]
246 self.ydata = [self.y(t, self.var_courbure, self.courb_finale) for t in
247 ↳ np.linspace(0, int(self.longueur), self.precision)]
248 self.patch = matplotlib.lines.Line2D(self.xdata, self.ydata, color='green')
249 self.patch.set_transform(matplotlib.transforms.Affine2D().rotate_deg_around(0, 0,
250 ↳ voiture.angle) + matplotlib.transforms.Affine2D().translate(voiture.x,
251 ↳ voiture.y) + self.axes.transData)
252
253 self.abscisse_curviligne_carre = 0
254 self.braquage = 0
255 self.V_lisse = 0
256
257 def x(self, t, var_courbure, courb_finale):
258     """
259     Abscisse à t de la clothoïde : intégrale de Fresnel.
260
261     Parameters
262     -----
263     t : numpy.float64
264         Borne de l'intégrale.
265     var_courbure : float
266         Variation de la courbure.
267     courb_finale : float
268         Courbure finale de la clothoïde.
269
270     Returns
271     -----
272     None.
273
274     """
275     return integrale(lambda s: np.cos(var_courbure*s**2 + courb_finale*s)/3, 0, t)[0]
276     ↳ Intégrale de Fresnel avec facteur 1/3 correctif (meilleure mise à l'échelle)
277
278 def y(self, t, var_courbure, courb_finale):
279     """
280     Ordonnée à t de la clothoïde : intégrale de Fresnel.
281
282     Parameters
283     -----
284     t : numpy.float64
285         Borne de l'intégrale.
286     var_courbure : float
287         Variation de la courbure.
288     courb_finale : float
289         Courbure finale de la clothoïde.
290
291     Returns

```

```

285 -----
286 None.
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
"""
return integrale(lambda s: np.sin(var_courbure*s**2 + courb_finale*s)/3, 0, t)[0] #
↳ Intégrale de Fresnel avec facteur 1/3 correctif (meilleure mise à l'échelle)

def get_parametric(self):
    """
    Permet d'obtenir la forme paramétrée de la tentacule. Récupère simplement les points
    car clothoïde paramétrée par définition puis transformée dans le plan.

    Returns
    -----
    chemin : numpy.ndarray
        Abscisses et ordonnées de la tentacule.
        Ex: numpy.array([[ x1,  y1],
                        [ x2,  y2],
                        [ x3,  y3]])

    """
    chemin = (self.patch.get_transform() -
    ↳ axes_simu.transData).transform_path(self.patch.get_path()).cleaned().vertices[:-1]
    return chemin

def __iter__(self):
    """
    Implémente l'itérabilité.
    """
    self.chemin = self.get_parametric()
    return self

def __next__(self):
    """
    Passe à l'élément suivant (la tentacule étant un itérable) : met à jour la forme
    paramétrée, et retourne les nouveaux états de la voiture.
    ↳

    Raises
    -----
    StopIteration
        Si la tentacule est parcourue en entier.

    Returns
    -----
    new_x : float
        Nouvelle abscisse.
    new_y : float
        Nouvelle ordonnée.
    dx : float
        Déplacement effectué en x.
    dy : float
        Déplacement effectué en y.
    new_angle : numpy.float64
        Nouvel angle de la voiture.
    braquage : numpy.float64 ou celui de la liste de braquage passée en paramètre lors
    ↳ de la définition de la tentacule.
        Nouveau braquage de la voiture.

    """
    try:
        new_x = self.chemin[1][0]
        new_y = self.chemin[1][1]

```

```

342     dx = new_x - self.chemin[0][0]
343     dy = new_y - self.chemin[0][1]
344     new_angle = np.degrees(np.arctan2(dy, dx))
345     self.abcisse_curviligne_carre += new_x**2 + new_y**2
346     braquage = self.var_courbure * np.sqrt(self.abcisse_curviligne_carre)
347     self.braquage = braquage
348     self.chemin = np.delete(self.chemin, 0, 0)
349     return new_x, new_y, dx, dy, new_angle, braquage
350 except IndexError:
351     raise StopIteration("Fin de la tentacule")
352
353
354 class Tentacule_Circulaire(_Tentacule):
355     """
356     Objet tentacule en forme de cercle, dépendant d'une voiture, et caractérisé par son
357     ↪ ordre et son angle de braquage.
358     """
359
360     def __init__(self, voiture, k, delta_k, axes):
361         super().__init__(axes, voiture)
362
363         self.numero = k
364         self.braquage = delta_k
365
366         self.L1 = 8 + 72 * (self.voiture.vitesse/self.voiture.vitesse_max)*1.2 # form
367         ↪ empirique, équation (9.5) p.123
368         # formules pour la longueur p.123 équation (9.4)
369         # formules pour le rayon de courbure p.122 équation (9.3)
370
371         if 1 <= k <= 40: # tentacule à orienter vers la droite
372             self.longueur = self.L1 + 20 * np.sqrt((k-1) / 40)
373             self.courbure = -np.tan(delta_k) / self.voiture.longueur
374
375             self.rayon = 1 / self.courbure
376             self.angle_rad = self.longueur / self.rayon
377             self.angle = self.angle_rad * 180 / np.pi
378
379             # generation de l'arc de cercle
380             self.angle_debut = 180 - self.angle
381             self.angle_fin = 180
382             centre_x = self.voiture.x + self.rayon / 2
383
384             self.signe = -1 # parcours de la tentacule dans le sens inverse, voir
385             ↪ Voiture.update_tentacule()
386
387         elif k == 41:
388             self.longueur = self.L1 + 20 * np.sqrt((k-1) / 40)
389             self.courbure = -np.tan(delta_k) / self.voiture.longueur
390
391             self.rayon = 1 / self.courbure
392             self.angle_rad = self.longueur / self.rayon
393             self.angle = self.angle_rad * 180 / np.pi
394
395             self.angle_debut = 180
396             self.angle_fin = 180 - self.angle
397             centre_x = self.voiture.x + self.rayon / 2
398
399             self.signe = 1 # parcours de la tentacule dans le bon sens, voir
400             ↪ Voiture.update_tentacule()
401
402         else: # 42<=k<=81 # tentacule à orienter vers la gauche
403             self.longueur = self.L1 + 20 * np.sqrt((81-k) / 40)

```

```

404     self.courbure = np.tan(delta_k) / voiture.longueur
405
406     self.rayon = 1 / self.courbure
407     self.angle_rad = self.longueur / self.rayon # en radian
408     self.angle = self.angle_rad * 180 / np.pi # en degrés
409
410     self.angle_debut = 0
411     self.angle_fin = self.angle
412     centre_x = self.voiture.x - self.rayon / 2
413
414     self.signe = 1 # parcours de la tentacule dans le bon sens, voir
415     ↪ Voiture.update_tentacule()
416
417     # On créé le dessin de la tentacule comme si la voiture était orientée vers le Nord
418     self.patch = matplotlib.patches.Arc((centre_x, self.voiture.y), self.rayon,
419     ↪ self.rayon, theta1=self.angle_debut, theta2=self.angle_fin, color='pink')
420     # On applique la rotation de la voiture au dessin de la tentacule
421
422     ↪ self.patch.set_transform(matplotlib.transforms.Affine2D().rotate_deg_around(self.v
423     ↪ self.voiture.y, self.voiture.angle - 90) + self.axes.transData)
424
425     self.V_lisse = abs(self.braquage - self.voiture.braquage) / (2 *
426     ↪ self.voiture.braquage_max)
427
428     def get_parametric(self):
429         """
430         Permet d'obtenir la forme paramétrée de la tentacule. Modifie le patch en
431         ↪ définissant un nombre de points explicite, et récupère les coordonnées.
432
433         Returns
434         -----
435         chemin : numpy.ndarray
436             Abscisses et ordonnées de la tentacule.
437             Ex: numpy.array([[ x1,   y1],
438                             [ x2,   y2],
439                             [ x3,   y3]])
440
441         """
442         # Définition d'un arc avec un nombre de points contrôlé
443         self.patch._path = matplotlib.patches.Path.arc(self.angle_debut, self.angle_fin,
444         ↪ self.precision)
445         # Paramétrisation de la tentacule
446         chemin = (self.patch.get_transform() -
447         ↪ axes_simu.transData).transform_path(self.patch.get_path()).cleaned().vertices[:-1]
448         return chemin
449
450     def __iter__(self):
451         """
452         Implémente l'itérabilité.
453         """
454         self.chemin = self.get_parametric()
455         return self
456
457     def __next__(self):
458         """
459         Passe à l'élément suivant (la tentacule étant un itérable) : met à jour la forme
460         ↪ paramétrée, et retourne les nouveaux états de la voiture.
461
462         Raises
463         -----
464         StopIteration
465             Si la tentacule est parcourue en entier.

```

```

453 Returns
454 -----
455 new_x : float
456     Nouvelle abscisse.
457 new_y : float
458     Nouvelle ordonnée.
459 dx : float
460     Déplacement effectué en x.
461 dy : float
462     Déplacement effectué en y.
463 new_angle : numpy.float64
464     Nouvel angle de la voiture.
465 braquage : numpy.float64 ou celui de la liste de braquage passée en paramètre l
466 ↪ de la définition de la tentacule.
467     Nouveau braquage de la voiture.
468
469 """
470 try:
471     new_x = self.chemin[1][0]
472     new_y = self.chemin[1][1]
473     dx = new_x - self.chemin[0][0]
474     dy = new_y - self.chemin[0][1]
475     new_angle = np.degrees(np.arctan2(dy, dx))
476     braquage = self.braquage
477     self.chemin = np.delete(self.chemin, 0, 0)
478     return new_x, new_y, dx, dy, new_angle, braquage
479 except IndexError:
480     raise StopIteration("Fin de la tentacule")
481
482 class Voiture:
483     """
484     Objet voiture, défini par sa forme. D'autres paramètres fixes sont définis après, on
485     ↪ pourra les mettre en paramètre plus tard si plusieurs voitures il y a.
486     """
487
488     def __init__(self, shape_voiture, axes, type_tentac=Tentacule_Circulaire,
489     ↪ nombre_tentac=80):
490         """
491         Parameters
492         -----
493         shape_voiture : matplotlib.patches.Rectangle
494             Forme de la voiture.
495         axes : matplotlib.axes._axes.Axes
496             Axes dans lesquels la voiture sera affichée.
497         type_tentac : __main__.Tentacule, optional
498             Le type de tentacule qui vont être générées par la voiture. The default is
499     ↪ Tentacule_Circulaire.
500         nombre_tentac : int, optional
501             Nombre de tentacules à générer. The default is 80.
502
503         Returns
504         -----
505         None.
506
507         """
508         self.shape_voiture = shape_voiture
509         self.shape_voiture.rotation_point = 'center'
510
511         self.shapely_poly =
512     ↪ shapely.geometry.Polygon((self.shape_voiture.get_transform()).transform_path(self
513
514         self.longueur = self.shape_voiture.get_height()
515         self.largeur = self.shape_voiture.get_width()
516         self.vitesse_max = VITESSE_MAX_VOIT
517         self.accel_lat_max = ACCEL_LAT_MAX
518
519         self.vitesse = VITESSE_INIT_VOIT
520         self.image_vitesse = self.vitesse
521         self.x, self.y = self.shape_voiture.get_center()
522         self.angle = INCLINAISON_VOITURE_INIT
523         self.braquage = 0
524         self.distance_collision = self.vitesse**2 / 2*self.accel_lat_max
525         self.courbure_max = self.accel_lat_max / self.vitesse ** 2 # formule p.122 équation
526     ↪ (9.1)
527         self.braquage_max = np.arctan(self.longueur * self.courbure_max) # formule p.122
528     ↪ équation (9.2)
529
530         self.type_tentac = type_tentac
531         self.nombre_tentac = nombre_tentac
532         self.tentacules = []
533
534         self.chemin = iter(()) # Chemin que la voiture a à suivre. Iterable d'une
535     ↪ tentacule, cf update_chemin
536
537         self.tps_debut = time.monotonic()
538
539         self.axes_simu, self.axes_vitesse, self.axes_braquage = axes
540
541         self.traceur = PETIT_POU CET
542         self.trace = matplotlib.lines.Line2D([], [])
543         self.courbe_vitesse = matplotlib.lines.Line2D([time.monotonic() - self.tps_debut],
544     ↪ [VITESSE_INIT_VOIT])
545         self.courbe_braquage = matplotlib.lines.Line2D([time.monotonic() - self.tps_debut],
546     ↪ [BRAQUAGE_VOLANT_INIT])
547         self.axes_simu.add_line(self.trace)
548         self.axes_vitesse.add_line(self.courbe_vitesse)
549         self.axes_braquage.add_line(self.courbe_braquage)
550         self.axes_vitesse.set_ylim(0, self.vitesse_max)
551         self.axes_braquage.set_ylim(-self.braquage_max, self.braquage_max)
552
553     def collision(self, objet):
554         """
555         Si la voiture cogne l'objet passé en argument, la simulation est mise en pause.
556
557         Parameters
558         -----
559         objet : shapely.geometry.polygon.Polygon
560             Forme de l'objet (obstacle ou voiture) issue du module shapely.
561
562         Returns
563         -----
564         None.
565
566         """
567         if self.shapely_poly.intersects(objet) is True:
568             print("COLLISION VOITURE ", id(self))
569             if SECURITE is True:
570                 print("Arrêt de la simulation")
571                 self.simu.pause_replay()
572

```

```

566 def deplace(self):
567     """
568     S'il existe un chemin (itérable de la tentacule suivie), déplace la voiture suiva
↪ ce chemin et met à jour les données.

569
570     Raises
571     -----
572     erreur
573     Le chemin à suivre n'est pas défini ou est terminé.
574
575     Returns
576     -----
577     None.
578
579     """
580     try:
581         self.x, self.y, dx, dy, self.angle, self.braquage = next(self.chemin)
582         tp = time.monotonic() - self.tps_debut
583         dt = tp - self.courbe_vitesse.get_data()[0][-1]
584         tps = np.append(self.courbe_vitesse.get_data()[0], tp)
585         # La "vitesse" ne change pas car normalement le véhicule garde une vitesse à
↪ près constante. On peut obtenir (tracé dans les datas) la vitesse du
↪ véhicule sur la simulation,
↪ qui nous indique si ce qu'on voit est fidèle à la réalité ou non (temps de
↪ latence trop grands et déplacements chaotiques par accoups)

586
587         self.shape_voiture.set(x=self.x - self.largeur/2, y=self.y - self.longueur/2,
↪ angle=self.angle)
588         self.shapely_poly = shapely.geometry.Polygon((self.shape_voiture.get_transfor
↪ m
↪ self.axes_simu.transData).transform_path(self.shape_voiture.get_path()).v

589
590         if self.traceur is True:
591             self.trace.set_data(np.append(self.trace.get_data()[0], self.x),
↪ np.append(self.trace.get_data()[1], self.y))
592             self.courbe_vitesse.set_data(tps, np.append(self.courbe_vitesse.get_data()[1]
↪ (dx**2 + dy**2)**0.5 / dt))
593             self.courbe_braquage.set_data(tps, np.append(self.courbe_braquage.get_data()[
↪ self.braquage))
594             self.axes_braquage.set_xlim(right=tp + 10)

595
596     except StopIteration:
597         print("TENTACULE ÉPUISÉE, ON NE SAIT PLUS QUEL CHEMIN SUIVRE")
598         print("Mise à jour forcée du chemin à suivre")
599         print("~ appel du gestionnaire de tentacules et coloration en marron ~")
600         self.simu.gestion_tentac(self)
601         for tentac in self.tentacules:
602             tentac.patch.set_linestyle(":")
603
604
605     def freiner(self):
606         """
607         Freine la voiture : divise la vitesse par 2, simulant un coup de frein (vitesse
↪ moyenne de 35km/h).
608         Arrête la simulation si la vitesse est <1m/s (on cale).
609
610     Returns
611     -----
612     None.
613
614     """
615     self.tentacule_suivie = self.choix_tentacule(self.tentacules) # TODO: modifier
↪ paramètre lambda avec critère dégagement de tentacule seulement (pas braquage

```

```

616 self.tentacule_suivie.patch.set(color="black", linewidth=5)
617 self.chemin = iter(self.tentacule_suivie)
618 self.vitesse /= 2
619 if self.vitesse < 0.5:
620     print("Tu as calé bg")
621     self.simu.pause_replay(stop=True)
622
623
624 def update_chemin(self):
625     """
626     Supprime les tentacules actuelles et en génère de nouvelles, puis met à jour le
↪ chemin que la voiture doit emprunter.
627
628     Returns
629     -----
630     None.
631
632     """
633     self.remove_tentacules()
634     self.generation_tentacules()
635
636     tentacules_navigables =
↪ self.get_tentacules_navigables_et_lisses(self.simu.obstacles_fixes +
↪ self.simu.obstacles_mvt + self.simu.voitures)
637 if tentacules_navigables == []: # les tentacules du milieu ne sont pas navigables :
↪ on va chercher les tentacules extrémales
638     print("Les tentacules lisses ne sont pas navigables")
639     tentacules_navigables =
↪ self.get_all_tentacules_navigables(self.simu.obstacles_fixes +
↪ self.simu.obstacles_mvt + self.simu.voitures)
640
641     try:
642         tentacule_a_suivre = self.choix_tentacule(tentacules_navigables)
643
644     except ValueError as err: # Pas de tentacule navigable
645         print(err)
646         print("Freinage !")
647         self.freiner()
648
649     else:
650         tentacule_a_suivre.patch.set_color("purple")
651         tentacule_a_suivre.patch.set_linewidth(3)
652         self.tentacule_suivie = tentacule_a_suivre
653         self.chemin = iter(tentacule_a_suivre)
654
655     def generation_tentacules(self):
656         """
657         Génère les 80 tentacules que la voiture aura la possibilité d'emprunter.
658
659     Returns
660     -----
661     None.
662
663     """
664     if self.type_tentac == Tentacule_Circulaire:
665         self.tentacules = []
666         deltas = np.linspace(-self.braquage_max, self.braquage_max, 80) # 80 est un
↪ nombre empirique de tentacules défini dans la thèse (il est bon compromis
↪ entre précision et temps de calcul)
667         for k in range(1, 81):
668             self.tentacules.append(self.type_tentac(self, k, deltas[k-1],
↪ self.axes_simu)) # Ajout de la tentacule aux tentacules appartenant au
↪ véhicule

```

```

668     for k in range(80):
669         self.axes_simu.add_patch(self.tentacules[k].patch) # Ajout de la tentacule
670         ↳ à la figure
671     elif self.type_tentac == Tentacule_clothoïdale:
672         self.tentacules = []
673         ro_max = self.accel_lat_max / self.vitesse**2
674         courbure_tentac_init = self.braquage * np.pi / 180
675         rap_min = 2*(-ro_max - courbure_tentac_init) / self.distance_collision
676         rap_max = 2*(+ro_max - courbure_tentac_init) / self.distance_collision
677         deltas = np.linspace(rap_min, rap_max, 40)
678         for rap in deltas:
679             self.tentacules.append(Tentacule_clothoïdale(self, courbure_tentac_init,
680                 ↳ rap, self.axes_simu))
681         for tentacule in self.tentacules:
682             self.axes_simu.add_line(tentacule.patch)
683     else:
684         pass # TODO: compléter avec tentac inconnue
685
686 def remove_tentacules(self):
687     """
688     Enlève les tentacules (qui appartiennent au véhicule) de la figure.
689
690     Returns
691     -----
692     None.
693
694     """
695     for tentacule in self.tentacules:
696         tentacule.patch.set(visible=False) # car si simple remove, on perd les
697         ↳ axes_simu de référence et c pas bon pour le blit...
698
699 def get_espace_collision(self):
700     """
701     Génère et stocke une zone de support pour le véhicule (distance de sécurité autour
702     ↳ de la voiture pour éviter quelconque collision).
703     cf von Hundelshausen et al 2008 p.650
704
705     Returns
706     -----
707     shapely.geometry.polygon.Polygon
708     La zone de support évitant les collisions.
709
710     """
711     self.distance_collision = self.vitesse**2 / 2*self.accel_lat_max
712     self.espace_collision = shapely.geometry.Point((self.x,
713         ↳ self.y)).buffer(self.distance_collision + self.longueur/2)
714     return self.espace_collision
715
716 def is_tentacule_navigable(self, tentacule, objets_alentours):
717     """
718     Teste si la tentacule spécifiée est navigable (cogne des objets alentours dans le
719     ↳ périmètre de sécurité). Méthode non utilisée.
720
721     Parameters
722     -----
723     tentacule : __main___.Tentacule
724         Tentacule à tester.
725     objets_alentours : __main___.Voiture ou __main___.Obstacle
726         Objets à éviter.
727
728     Returns
729     -----

```

```

724     bool
725     True si la tentacule est navigable, False sinon.
726
727     """
728     return tentacule.is_navigable(objets_alentours)
729
730 def choix_tentacule(self, tentacules_navigables, ponderation_meilleure_tentac=lambda
731     ↳ tentac: tentac.V_lisse**4 + tentac.braquage**2):
732     """
733     Choisi une tentacule à suivre.
734
735     Parameters
736     -----
737     tentacules_navigables : list de __main___.Tentacule
738         Tentacules dispo à trier pour choisir.
739     ponderation_meilleure_tentac : callable, optional
740         Fonction de tri. The default is lambda tentac: tentac.V_lisse**4 +
741         ↳ tentac.braquage**2.
742
743     Raises
744     -----
745     ValueError
746         S'il n'y a aucune tentacule navigable (tentacules_navigables = [] par exemple).
747
748     Returns
749     -----
750     __main___.Tentacule
751         Tentacule à suivre (meilleure parmi celles données en argument).
752
753     """
754     # TODO : faire un critère de dégagement du tentacule (p. 125 section 9.2.4.1)
755     # TODO moins important car nécessite GPS et tt (pas forcément notre objet d'étude)
756     ↳ faire un critère de rapprochement de la trajectoire globale (p.126 section
757     ↳ 9.2.4.3)
758
759     try:
760         self.tentacule_suivie = min(tentacules_navigables,
761             ↳ key=ponderation_meilleure_tentac) # On trie selon une loi empirique
762             ↳ fonction de V_lisse (écart au braquage actuel) et de braquage (trajectoire
763             ↳ droite). Puis on prend le min
764         return self.tentacule_suivie
765     except ValueError:
766         raise ValueError("AUCUNE TENTACULE NAVIGABLE")
767
768 def get_all_tentacules_navigables(self, objets_alentours):
769     """
770     Permet d'obtenir toutes les tentacules navigables.
771
772     Parameters
773     -----
774     objets_alentours : __main___.Voiture ou __main___.Obstacle
775         Objets à éviter.
776
777     Returns
778     -----
779     tentacules_navigables : list of __main___.Tentacule
780         Liste des tentacules navigables et que la voiture peut emprunter.
781
782     """
783     tentacules_navigables = []
784     for tentacule in self.tentacules:
785         if tentacule.is_navigable(objets_alentours):

```

```

779         tentacule.patch.set_color('g')
780         tentacules_navigables.append(tentacule)
781     return tentacules_navigables
782
783 def _simu(self, simu):
784     """
785     À ne pas appeler directement. Permet de définir la simulation dans laquelle évolue
786     ↪ la voiture.
787
788     Parameters
789     -----
790     simu : TYPE
791         DESCRIPTION.
792
793     Returns
794     -----
795     None.
796
797     """
798     self.simu = simu
799
800 def get_tentacules_navigables_et_lisses(self, objets_alentours):
801     """
802     Permet d'obtenir toutes les tentacules navigables et 'lisses' ie dont la variation
803     ↪ de braquage demandée n'est pas trop forte (<CONE_BRAQUAGE).
804
805     Parameters
806     -----
807     objets_alentours : __main__.Voiture ou __main__.Obstacle
808         Objets à éviter.
809
810     Returns
811     -----
812     tentacules_navigables : list of __main__.Tentacule
813         Liste des tentacules navigables et que la voiture peut emprunter.
814
815     """
816     tentacules_lisses_et_navigables = []
817     for tentacule in [tentac for tentac in self.tentacules if abs(tentac.braquage -
818     ↪ self.braquage) < CONE_BRAQUAGE]:
819         if tentacule.is_navigable(objets_alentours):
820             tentacule.patch.set_color('g')
821             tentacules_lisses_et_navigables.append(tentacule)
822     return tentacules_lisses_et_navigables
823
824 class Obstacle():
825     """
826     Instance commune à tous type d'obstacle. Réunit des méthodes et attributs universels.
827     ↪ pas appeler directement, est automatiquement appelée par d'autres instances.
828     """
829
830 def __init__(self, patch):
831     """
832     Parameters
833     -----
834     patch : matplotlib.patches.Rectangle
835         Forme de l'obstacle.
836
837     Returns
838     -----
839     None.
840
841     """
842
843     self.patch = patch
844     self.x, self.y = self.patch.get_center()
845     self.shapely_poly =
846     ↪ shapely.geometry.Polygon((self.patch.get_transform()).transform_path(self.patch.g
847
848 class Obstacle_fixe(Obstacle):
849     """
850     Instance pour un obstacle qui ne bougera pas au cours du temps.
851     """
852
853 def __init__(self, patch):
854     """
855     Parameters
856     -----
857     patch : matplotlib.patches.Rectangle
858         Forme de l'obstacle.
859
860     Returns
861     -----
862     None.
863
864     """
865     super().__init__(patch)
866
867 class Obstacle_mvt():
868     """
869     Instance pour un obstacle qui est amené à bouger au cours du temps. Fonctionnalité
870     ↪ encore non développée correctement.
871     """
872
873 def __init__(self):
874     pass
875
876 class Simulation_Ref_Terre:
877     """
878     Classe qui gère tout l'aspect simulation : animation + gestion des objets.
879
880     Parameters
881     -----
882     voitures: list
883         Liste des voitures présentes
884     obstacles: list # TODO
885         liste des obstacles FIXES. Ils ne seront pas animés
886     obstacles_mvt: list # TODO
887         liste des obstacles à déplacer. Fonction développée plus tard
888     """
889
890 def __init__(self, voitures=[], obstacles_fixes=[], obstacles_mvt=[], animations=[],
891     ↪ timers=[]):
892     """
893     Parameters
894     -----
895     voitures : list of __main__.Voiture, optional
896         Voitures dans la simulation. The default is [].
897     obstacles_fixes : list of __main__.Obstacle_fixe, optional
898         Obstacles fixes dans la simulation. The default is [].
899     obstacles_mvt : list of __main__.Obstacle_mvt, optional
900

```

```

996 Obstacles en mouvement dans la simulation. The default is [].
997 animations : list of matplotlib.animation.FuncAnimation, optional
998 Animations qui gèrent l'affichage et l'interaction. The default is [].
999 timers : list of matplotlib.backends.backend_qt.TimerQT, optional
1000 Timers qui gèrent l'animation et l'interaction. The default is [].
1001
1002 Returns
1003 -----
1004 None.
1005
1006 """
1007 self.paused = False
1008
1009 self.animations = animations
1010 self.timers = timers
1011
1012 self.voitures = voitures
1013 self.obstacles_fixes = obstacles_fixes
1014 self.obstacles_mvt = obstacles_mvt
1015 self.new_tentacules = [] # liste des tentacules à afficher (blit)
1016 self.old_tentacules = [] # liste des tentacules qui ne sont plus d'actualité, mai.
    ↳ qui doivent être supprimées (blit)
1017
1018 for voitures in self.voitures:
1019     voiture._simu(self)
1020     self.gestion_tentac(voiture) # créé les tentacules de la voiture, et créé un
    ↳ chemin à suivre (surement inexistant jusqu'alors...)
1021
1022 self.commentaires = COMMENTAIRES
1023
1024 def animer(self, i):
1025     """
1026     Anime la figure. S'occupe de déplacer les voitures et les obstacles.
1027     Note : fait appel au "task-manager des tentacules" pour savoir quoi actualiser dan.
    ↳ la figure (blit)
1028
1029     Parameters
1030     -----
1031     i : int
1032         Numéro d'appel de la fonction (elle est censée être appelée itérativement par
    ↳ une matplotlib.animation.
1033
1034     Returns
1035     -----
1036     list of artists
1037         Liste des dessins à actualiser (blit).
1038
1039     """
1040     # Déplace les voitures
1041     for voiture in self.voitures:
1042         voiture.deplace()
1043     # Déplace les obstacles
1044     for obstacle in self.obstacles_mvt:
1045         obstacle.deplace()
1046     try:
1047         # Retourne la liste des dessins à actualiser dans la figure
1048         # TODO : ajouter les obstacles lors de l'import de la fonctionnalité
1049         return [item for item in [voiture.shape_voiture, voiture.trace,
    ↳ voiture.courbe_vitesse, voiture.courbe_braquage] for voiture in
    ↳ self.voitures] + [objet.patch for objet in self.old_tentacules +
    ↳ self.new_tentacules + self.obstacles_fixes + self.obstacles_mvt]
1050     finally:
1051
1052 # Met juste après les valeurs adéquates à jour pour ne plus avoir à redessiner
    ↳ des dessins fixes (blit)
1053 self.old_tentacules = [] # Suppression des tentacules obsolètes, qu'on a bien
    ↳ effacé avec le return
1054 for tentac in self.old_tentacules:
1055     tentac.patch.remove()
1056 for tentac in self.new_tentacules:
1057     tentac.patch.set_animated(False) # Dés-anime les nouvelles tentacules
    ↳ (affichées avec le return) : elles ne doivent plus bouger jusqu'à une
    ↳ redéfinition de celles-ci. Sert ?
1058
1059 def gestion_tentac(self, voiture):
1060     """
1061     "Task-manager des tentacules"... Gère l'ajout et la suppression des tentacules
    ↳ (appartenant à la voiture *voiture*) de la figure.
1062
1063     Parameters
1064     -----
1065     voiture : Voiture
1066         objet voiture.
1067
1068     Returns
1069     -----
1070     None.
1071
1072     """
1073     for tentac in voiture.tentacules: # pour mettre à jour et enlever les tentacules
    ↳ qui vont être supprimées
1074         tentac.patch.set_animated(True) # Anime les tentacules à supprimer (elles
    ↳ étaient dés-animées) (blit)
1075         self.old_tentacules.append(tentac) # ajout à la liste des dessins à actualiser
    ↳ (blit)
1076
1077 voiture.update_chemin() # suppression du système actuel de tentacules pour la
    ↳ voiture, puis génération de nouvelles tentacules pour la voiture
1078
1079 for tentac in voiture.tentacules: # pour mettre à jour et ajouter les tentacules
    ↳ qui vont être ajoutées
1080     tentac.patch.set_animated(True) # anime les tentacules à ajouter (blit)
1081     self.new_tentacules.append(tentac) # ajout à la liste des dessins à actualiser
    ↳ (blit)
1082
1083 def collisions(self):
1084     """
1085     Teste les collisions pour chaque voiture.
1086
1087     Returns
1088     -----
1089     None.
1090
1091     """
1092     for voiture in self.voitures:
1093         for obstacle in self.obstacles_fixes:
1094             voiture.collision(obstacle.shapely_poly)
1095         for obstacle in self.obstacles_mvt:
1096             voiture.collision(obstacle.shapely_poly)
1097         for voit in self.voitures:
1098             if voit != voiture: # pas de collision qd la voiture se touche elle-même
1099                 voiture.collision(voit.shapely_poly)
1100
1101 def pause_replay(self, stop=False):
1102     """

```

```

1002 Met en pause ou remet en play.
1003
1004 Returns
1005 -----
1006 None.
1007
1008 """
1009 if self.paused:
1010     print("PLAY")
1011     for anim in self.animations:
1012         anim.resume()
1013     for timer in self.timers:
1014         timer.start()
1015     self.paused = False
1016 else:
1017     print("PAUSE")
1018     for anim in self.animations:
1019         anim.pause()
1020     for timer in self.timers:
1021         timer.stop()
1022     """
1023
1024     if stop is False:
1025         self.paused = True # Si juste une pause on le fait savoir, si c'est un
1026 ↪ stop, on fait croire que c'est encore la simu (pour que qd on replay ça replay pas)
1027     """
1028     self.paused = True
1029
1030 def ajout_anim(self, anim):
1031     """
1032     Ajoute une animation.
1033
1034     Parameters
1035     -----
1036     anim : matplotlib.animation.FuncAnimation
1037         Animation à ajouter.
1038
1039     Returns
1040     -----
1041     None.
1042
1043     """
1044     self.animations.append(anim)
1045
1046 def ajout_timer(self, timer):
1047     """
1048     Ajoute un timer.
1049
1050     Parameters
1051     -----
1052     timer : matplotlib.backends.backend_qt.TimerQT
1053         Timer à ajouter.
1054
1055     Returns
1056     -----
1057     None.
1058
1059     """
1060     self.timers.append(timer)
1061
1062 # création figure + axes
1063 figure_ref_terre, axes_simu = plt.subplots()
1064
1065 figure_data, (axes_vitesse, axes_braquage) = plt.subplots(2, 1, sharex=True)
1066 # titre
1067 figure_ref_terre.suptitle("Animation")
1068 figure_data.suptitle("Données en temps réel")
1069 axes_vitesse.set_title("Vitesse")
1070 axes_braquage.set_title("Braquage")
1071 axes_braquage.set_xlabel("Temps")
1072
1073 # création de la forme de la voiture
1074 shape_voiture = matplotlib.patches.Rectangle(POSITION_VOITURE_INIT, LONGUEUR_VEHC,
1075 ↪ LARGEUR_VEHC, fill=False, zorder=float('inf'))
1076 shape_voiture.rotation_point = 'center'
1077 shape_voiture.set_angle(INCLINAISON_VOITURE_INIT)
1078
1079 # création de la voiture
1080 voiture = Voiture(shape_voiture, [axes_simu, axes_vitesse, axes_braquage],
1081 ↪ Tentacule_clothoïdaire)
1082
1083 # ajout du dessin de la voiture à la figure
1084 axes_simu.add_patch(voiture.shape_voiture)
1085
1086 # création de la forme de l'obstacle
1087 shape_obstacle = matplotlib.patches.Rectangle((30, -2.5), 2, 1.5, fill=True,
1088 ↪ zorder=float('inf'), color='cyan')
1089 obstacle1 = Obstacle_fixe(shape_obstacle)
1090 axes_simu.add_patch(obstacle1.patch)
1091
1092 # création des bords de route
1093 shape_route_haut = matplotlib.patches.Rectangle((-10, 5), 90, 2)
1094 # shape_route_milieu = matplotlib.patches.Rectangle((-30, 0), 190, 2)
1095 shape_route_bas = matplotlib.patches.Rectangle((-10, -5), 90, 2)
1096 route_haut = Obstacle_fixe(shape_route_haut)
1097 route_milieu = Obstacle_fixe(shape_route_milieu)
1098 route_bas = Obstacle_fixe(shape_route_bas)
1099 axes_simu.add_patch(route_bas.patch)
1100 # axes_simu.add_patch(route_milieu.patch)
1101 axes_simu.add_patch(route_haut.patch)
1102
1103 # création de la simulation
1104 simulation = Simulation_Ref_Terre([voiture], [obstacle1, route_bas, route_haut])
1105
1106 # création et lancement de l'animation
1107 anim = matplotlib.animation.FuncAnimation(figure_ref_terre,
1108 ↪ simulation.animer,
1109 ↪ init_func=lambda: [voiture.shape_voiture for
1110 ↪ voiture in simulation.voitures] +
1111 ↪ [tentacule.patch for tentacule in
1112 ↪ simulation.new_tentacules] + [tentacule.patch
1113 ↪ for tentacule in simulation.old_tentacules],
1114 ↪ fargs=[],
1115 ↪ interval=80,
1116 ↪ cache_frame_data=False,
1117 ↪ blit=True,
1118 ↪ repeat=True)
1119
1120 # création d'un timer
1121 timer = figure_ref_terre.canvas.new_timer(interval=TEMPS_REAC)
1122 if SECURITE is not False:
1123     timer2 = figure_ref_terre.canvas.new_timer(40)
1124     timer2.add_callback(simulation.collisions)
1125     timer2.start()

```

```
1118     simulation.ajout_timer(timer2)
1119     figure_ref_terre.canvas.mpl_connect('close_event', lambda event: timer2.stop())
1120     # ajout d'une fonction appelée itérativement
1121     timer.add_callback(simulation.gestion_tentac, voiture)
1122     # démarrage du timer
1123     timer.start()
1124
1125     simulation.ajout_anim(anim)
1126     simulation.ajout_timer(timer)
1127     # stop le timer si on ferme la figure (pour éviter qu'une fonction continue de tourner en
1128     ↪ boucle sans intérêt une fois tout terminé, c'est plutôt mal vu...!)
1129     figure_ref_terre.canvas.mpl_connect('close_event', lambda event: timer.stop())
1129     figure_ref_terre.canvas.mpl_connect('key_press_event', lambda event: (print("Key Pressed"),
1130     ↪ simulation.pause_replay()))
1131
1132     axes_simu.autoscale()
1133     # axes_simu.set(xlim=(-50, 50), ylim=(-50, 150))
1134     axes_simu.axis('equal')
1135     plt.show()
```